19th April: Introduction to Web Sockets And OIDC continued

– Jeani

1

⇒ node-jose: is used to create keys in OIDC. It is a node JS library used to generate, manage and use keys to sign, verify and expose JWTs via JWKS in OIDC.

OIDC Implementation:

cert.js → reads the public and private key

1. Create a health route
2. Create a route ("./well-known/open-id configuration")
   2.1 ~~Fetch~~ Define ISSUER = ... //localhost: ${PORT}`,
   2.2 Then return JSON with
   authorization_endpoint
   userinfo_endpoint
   jwks_uri
3. Now we expose our public key at endpoint ("./well-known/jwks.json")
   → So when a client / backend recieves a request from user
   → It calls the "jwks.json" endpoint
   → gets the public key
   → uses it to verify the token
4. Now make a authentication endpoint ("/o/authenticate", ...)
   → Now we sends some html that open a page where user logs in
   → authenticate.html is served here

5. Now user submits login form at POST /o/authenticate/sign-in

Now we created a route where user sends their email and password.

5.1 First validate the input

Check if user exists by finding its email

Then compare the hashed password

5.2 Generate the ID token (signed using private key)

Then redirect back with authorization code

then exchange of token happens.

6. Client calls userinfo at GET "/o/userinfo"

Assignment : Create a POST route "/o/tokeninfo" where

→ client secret se application fetch kro

→ code ko validate kro

→ code jiss user ka hai uska access token refresh token

→ return {token}

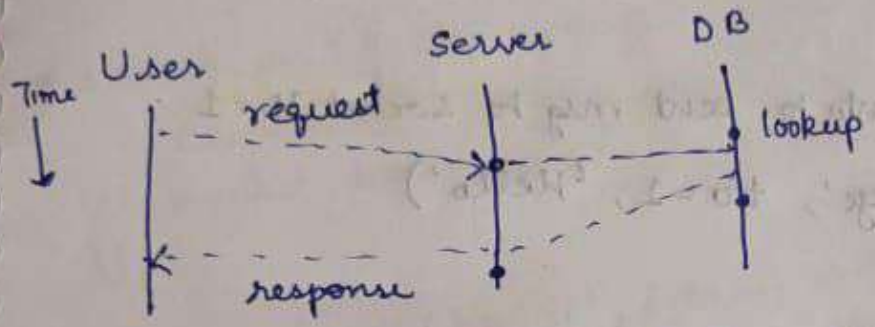
Basically to create an auth system like Clerk, where people can login using our system. Extend existing code. Our own OAuth 2.0

In step 6: at GET : /o/userinfo

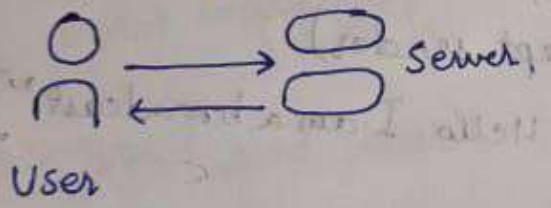
- Client sends using : Authorization : Bearer <token>
- Extract token from header
- Verify token (check if valid, not expired and matches the signature from JWKS
- Fetch user data

WebSockets:

→ Server : It serves something
while (True){
Always listening to requests
}



User opens the request and also closes the request It is one-way



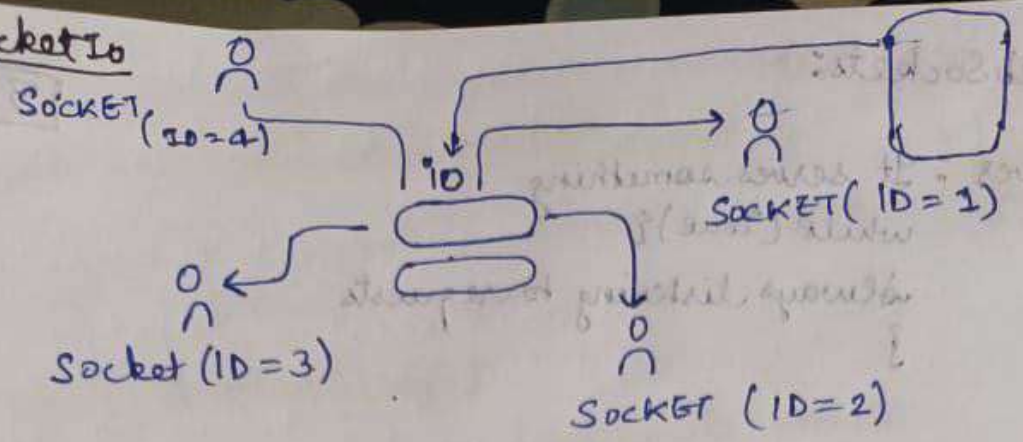
WebSockets (having duplex)

Meaning data can flow both ways (client ⇄ server) at the same time.

Socket IO (a library built on top of WebSockets)

Adds an abstraction layers
Gives features like
→ events
→ Rooms etc etc
→ Broadcast

Socket Io



Now socket 4, wants to sent msg to socket ID = 1

```
io.emit('message', to=1, 'Hello')
```

To send to everyone

```
io.emit("Hello everyone")
```

To broadcast (everyone except sender)

```
socket.broadcast.emit('Hello. I am a broadcast')
```

To create own socket-chat-app

1. Do backend setup for chat app

```
const server = http.createServer();
const io = new server();
io.attach(server); [binds Websockets layer]

server.listen(9000, () => {
    ....
});
```

2. A new handler is written a backend,

io.on('connection', (socket) => {

//So everytime a user connects,
each socket has its unique socketid,
console.log("A new socket is created/connected, socket.id);

→ socket.on('user:message', (data) => {
console.log('Message from socket', data);
socket.broadcast.emit('server:message', data);
});

⇒ socket.on('user:typing', (data) ⇒ {
console.log('User is typing', socket.id, data);
socket.broadcast.emit('server:user:typing', { id: socket.id});

Note: Don't send 'data.text'

1. For connection:
io.on("connection", (socket) ⇒ { })

2. Listening for incoming events.
Socket.on("eventName", (data) ⇒ { })

→ To send msg to a specific user/socket
Socket.emit("eventName", data)

⇒ To send to all clients
io.emit("eventName", data)

⇒ To broadcast to everyone except the sender

```
socket.broadcast.emit("eventName", data)
```

⇒ To send to a room

```
io.to("roomName").emit("eventName", data)
```

⇒ To disconnect

```
socket.on("disconnect", () => {})
```

On Client Side

⇒ To sent an event

```
socket.emit("eventName", data)
```

⇒ To receive event

```
socket.on("eventName", (data) => {})
```

⇒ To disconnect manually

```
socket.disconnect()
```

⇒ To connect

```
socket.connect()
```

So now we know,
when we send a message,

```
socket.emit("user:message", message)
```

Now at server,

```
socket.on("user:message", (msg) => {
    io.emit("server:message", msg)
```

Now client recieves,

```
socket.on("server:message", (msg) {
}
```

Broadcast is actually scary as,

This has a N^2 Fan Out Problem : If you have 100 users in a room,

1 message is sent to 99 users, so 100 users sending would be 9900 operations, so at scale its scary.

⇒ Data here is known as buffer (as data can be anything)

⇒ To make 100K connection, just socket connection takes 4 GB (mostly idle state)

How many process can you run on linux? 1024

port range : 35,000 - 60,000

Each connection has its unique IP + port

ulimit : controls the resource for a process. This is used to prevent a program from exhausting system resources.

This has a soft and hard limit

soft limit : current usable limit

hard limit : maximum allowed limit

Note : In Linux, OS treats everything as a file.

FD (File Descripter) : A number OS gives to our open resource.

In convex, DB is your state.

⇐ Assignment.

1. Create 10k checkboxes
2. Add event listeners to each checkbox
3. Add websockets to each checkbox
4. Make it public

Assignment: 1. Try making 1 million checkboxes.

2. Write a article on Linux file system.

Follow: @jaaaani404